

The Cruicality Of Shor's Algorithm In Breaking RSA Encryption

Bilisuma J. Merga

Abstract

Quantum computers using qubits instead of bits can do complex calculation in minutes, which can take years for classical computers. Quantum bits or qubits are, unlike bits, can have both 1 and 0 state at the same time. This state is called superposition and it is one properties of particles such as electrons in quantum mechanics. RSA is sufficiently safe cryptosystem algorithm encryption. It uses the fact that finding the prime factors of extremely large numbers(from 512 bits to 1024 bits and above). Shor's algorithm, founded by Peter Shor, is an algorithm used to find prime factors of large numbers easily and efficiently which makes it threat to our existing cryptocurrency. Even though know we have imperfect and small number of quantum computers, using this algorithm quantum computers easily break RSA. In this paper presented how RSA encryption works, and how do really quantum computers use Shor's algorithm to break RSA and the steps in the Shor's algorithm.

Key words= public key, private key, modular exponentiation period, coprime numbers, greatest common factor(gcf), least common multiple(lcm), shor's algorithm, modulus of numbers, Hadmard gate

The Cruiality Of Shor's Algorithm In Breaking RSA

1.Introduction

RSA was first publicly described in 1977 by Ron Rivest, Adi Shamir and Leonard Adleman of the Massachusetts Institute of Technology. Public key cryptography also known as asymmetric cryptography uses two different but mathematically linked keys: public key and private key. Generating both private and public keys are the most difficult part of RSA algorithm because both keys use very large numbers which makes it difficult for classical computers to work on. Granted, the emerging of quantum computers are threat to this algorithm. Even though today we are not using much of quantum computers, in the future they have the ability not only to invade our current technologies.

Quantum computers, like classical computers, uses different algorithms to solve different problems. One of well known is Shor's algorithm which is very crucial in finding prime factors of large numbers such as the one used in asymmetric cryptography. Shor's algorithm uses the quantum phase estimation method to find the period of modular exponentiation. The period is used to find numbers that most probably be the factors of the large prime number.

2.RSA Encryption

RSA got its name from the three genius computer scientists: R=Ron Rivest, S=Adi Shamir and A=Leonard Adleman. It is an incredibly useful asymmetric encryption algorithm. Its bases are the hardness of finding prime factorization of extremely large numbers. Briefly RSA encryption works:

1) Let's a and b two large prime numbers. We can check weather a and b are prime or not using probabilistic primality test algorithm such as:

Fermat Primality Test: for coprime numbers p and a the following equation holds true

$$a^p \equiv a \pmod{p} \Rightarrow a^{np+1} \equiv a \pmod{p}, \text{ Where } n \text{ is any integer}$$

Miller-Rebin primality test.

2) multiply both a and b to form the public key and private key modulus, N .

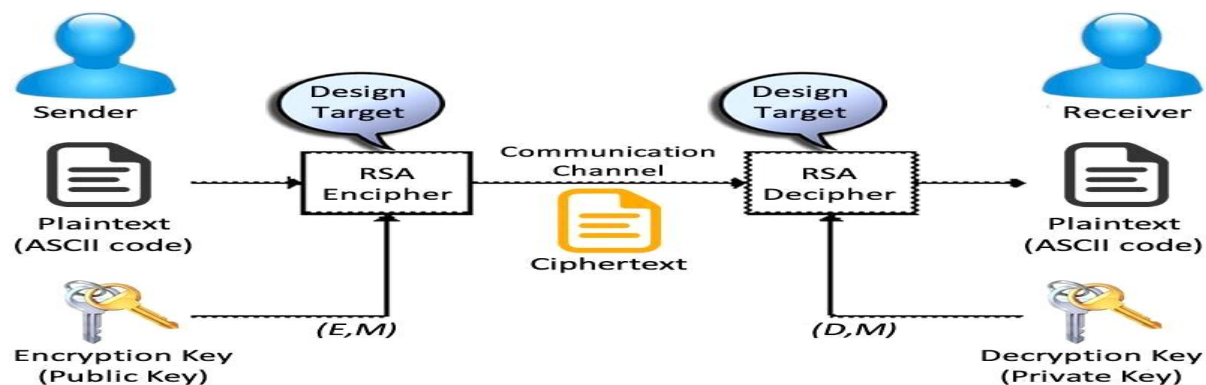
$$N = a.b$$

Pubic key: a large numerical value that is used to encrypt data. The key can be generated by a software program, but more often it is provided by a trusted designated authority with accessible direction and made available to everyone through public.

Private key: A private key is an alphanumeric code generated by a cryptocurrency wallet. It is used to authorize transactions and prove ownership of a blockchain asset.

To easily know how it works, see the picture below.

The Cruiality Of Shor's Algorithm In Breaking RSA



3) Compute the Euler Totient function ($\phi(N)$)

$\phi(N)$ counts the numbers of integer between 1 and N , which are coprime to prime N (does not have common factor with N).

If $N = a.b$, $\phi(N) = (a-1)(b-1)$

Where a and b are prime numbers

4) Choose any integer c such that c is coprime to $\phi(N)$, that is $\text{GCF}(C, \phi(N)) = 1$.

Tips: As our $\phi(N)$ is very large, we want C to be also very large because it is safer and difficult to get.

5) Compute other integer d , such that

$$c.d = 1 \pmod{\phi(N)}$$

6) The pair of positive integers (c, N) is public key.

7) The pair of positive integers (d, N) is private key.

Ciphered message = $(\text{message})^c \pmod{N}$

Deciphered message = $(\text{ciphered message})^d \pmod{N}$

To get insights see the example below.

Example of RSA Encryption

- Let us $a = 5$ and $b = 7$
- Public key modulus $N = 5 * 7 = 35$
- Euler Totient function $\Rightarrow \phi(N) = (5 - 1)(7 - 1) = 4 * 6 = 24$
- Any integer c coprime to $\phi(N) \Rightarrow$ let us say $C = 11$
- To compute d , $11d = 1 \pmod{24}$
 $11d - 1 = 0 \pmod{24}$
 $11d - 1 = n24$, for $n = 1$, $d = 25/11$, for $n = 2$, $d = 4$, for $n = 3$, $d = 70/11$, for $n = 5$
 $d = 11$
- Public key $\Rightarrow (11, 35)$
- Private key $\Rightarrow (11, 35)$

Let the message be $A = 5$,

Encrypted message = $5^{11} \pmod{35} = 10$

To decrypt the message = $10^{11} \pmod{35} = 5$

$S = A \Rightarrow$ The message is accepted to the one with the right private key

The Cruiality Of Shor's Algorithm In Breaking RSA

Generally, private keys are found only with the receiver and the receiver can decrypt the message.

3.How do quantum computers break RSA by finding prime factors of large number N.

As we previously said public keys(c, N) are shared public, but only the right receiver has private key(d, N). Someone can easily get encrypted message and the number N (up to 1024 bit). However, to decrypt the cypher-text we need both d and N . To find d we need the prime factors. Quantum computers using Shor's algorithm can easily find the prime factors.

Shor's Algorithm

Let $N=a.b$, pick a number g , such that g is a non-trivial square root of 1 modulus N .

$$g^2=1(\text{mod } N) \quad \text{and } g \neq \pm 1(\text{mod } N)$$

$$g^2 - 1 = 0(\text{mod } N)$$

$$(g-1)(g+1) = 0(\text{mod } N)$$

Steps used in shor's algorithm:

1. Choose any positive integer g , less than N , and check whether it has common factor with N or not. In this condition we have two cases.

Case 1: if g and N has some common factors, we have good chance of finding one of prime factors of N .

Case 2: if g and N are coprime numbers: $\text{gcf}(N, g) = 1$, we follow the next steps,

Recall that multiplicative group $(\mathbb{Z}/p\mathbb{Z})^\times$, the group of

all congruence classes relatively prime to p , is a cyclic group; that is, there is some $g \in (\mathbb{Z}/p\mathbb{Z})^\times$ that generates the group. Thus, for every $a \in (\mathbb{Z}/p\mathbb{Z})^\times$, we can write

$$a = g^r (\text{mod } p) :$$

Euler's theorem tells us that $a^{p-1} = 1(\text{mod } p)$. So, there exists a smallest integer r such that $a^r = 1(\text{mod } p)$. This value is also called the order of a .

From this find the period, r , of the function $f(x) = g^x(\text{mod } N)$, the period of this function indicates , $g^r = 1(\text{mod } N)$

2.If r is even, we let $m = g^{r/2}$

After saying this $(m+1)(m-1) = 0(\text{mod } N)$ we can find numbers possibly have factors with N and not equal to N , if not we go back to step 1, start it over.

3. If step 2 is successful, we use the factor we get and number N to find the other number.

4. If we find both numbers a and b we use it to find (d, N) which is private key.

How does Shor's algorithm simplify the finding of period r ?

Quantum computers factorize the number N easily by find the period r of the function $f(x) = g^x(\text{mod } N)$.

As we said earlier, $(g^{r/2} + 1)(g^{r/2} - 1) = sN$, where s is any positive integer,

The Cruiality Of Shor's Algorithm In Breaking RSA

But $N=a.b \Rightarrow \text{gcf}(g^{r/2}+1, N)$ and $\text{gcf}(g^{r/2}-1, N)$

Both the gcfs have high probability of being the prime factors

Steps to find period r :

1. We need to set up two registers. The second(work) register uses n qubits, where n is the smallest integer such that $N \leq 2^n$, i.e., $n = \lceil \log_2 N \rceil$. The size of the first(control) register determines how accurate of an approximation the circuit produces. It can be shown that using $m=2n+1$ qubits give sufficient accuracy to find r . The exact quantum circuit depends on the parameters r and N , which define the problem.

2. Using quantum phase estimation with unitary U and input state $|0\rangle^{\otimes(2n+1)}|1\rangle$ in the work or second register.

The unitary U gives $xa \pmod N$ for any input N

$U_{a,N}|x\rangle = |xa \pmod N\rangle$

To illustrate this,

$$U_a^0, N|1\rangle = |1 \pmod N\rangle$$

$$U_a^1, N|1\rangle = |a \pmod N\rangle$$

$$U_a^2, N|1\rangle = |a^2 \pmod N\rangle \dots\dots\dots$$

$$U_a^r, N|1\rangle = |a^r \pmod N\rangle = |1 \pmod N\rangle$$

Let's say we have constructed many qubit states $|U_s\rangle$ in the seond register.

$$|U_s\rangle = \frac{1}{\sqrt{r}} \left(e^{\frac{-2\pi i s(0)}{r}} |1 \pmod N\rangle + e^{\frac{-2\pi i s(2)}{r}} |a \pmod N\rangle + \dots + e^{\frac{-2\pi i s(r-2)}{r}} |a^{r-2} \pmod N\rangle + e^{\frac{-2\pi i s(r-1)}{r}} |a^{r-1} \pmod N\rangle \right)$$

If we apply U gate to the state:

$$\begin{aligned} U|U_s\rangle &= U \cdot \frac{1}{\sqrt{r}} \left(e^{\frac{-2\pi i s(0)}{r}} |1 \pmod N\rangle + e^{\frac{-2\pi i s(1)}{r}} |a \pmod N\rangle + \dots + e^{\frac{-2\pi i s(r-1)}{r}} |a^{r-1} \pmod N\rangle \right) \\ &= \frac{1}{\sqrt{r}} \left(e^{\frac{-2\pi i s(0)}{r}} |a \pmod N\rangle + e^{\frac{-2\pi i s(1)}{r}} |a^2 \pmod N\rangle + \dots + e^{\frac{-2\pi i s(r-1)}{r}} |a^r \pmod N\rangle \right) \end{aligned}$$

Using the concept of eigenvalue

$$\begin{aligned} U|U_s\rangle &= e^{\frac{2\pi i s}{r}} \cdot e^{\frac{-2\pi i s}{r}} \cdot \frac{1}{\sqrt{r}} \left(e^{\frac{-2\pi i s(0)}{r}} |a \pmod N\rangle + e^{\frac{-2\pi i s(1)}{r}} |a^2 \pmod N\rangle + \dots + e^{\frac{-2\pi i s(r)}{r}} |a^r \pmod N\rangle \right) \\ &= \frac{1}{\sqrt{r}} \cdot \left(e^{\frac{-2\pi i s(1)}{r}} |a \pmod N\rangle + e^{\frac{-2\pi i s(2)}{r}} |a^2 \pmod N\rangle + \dots + e^{\frac{-2\pi i s(r)}{r}} |a^r \pmod N\rangle \right) \end{aligned}$$

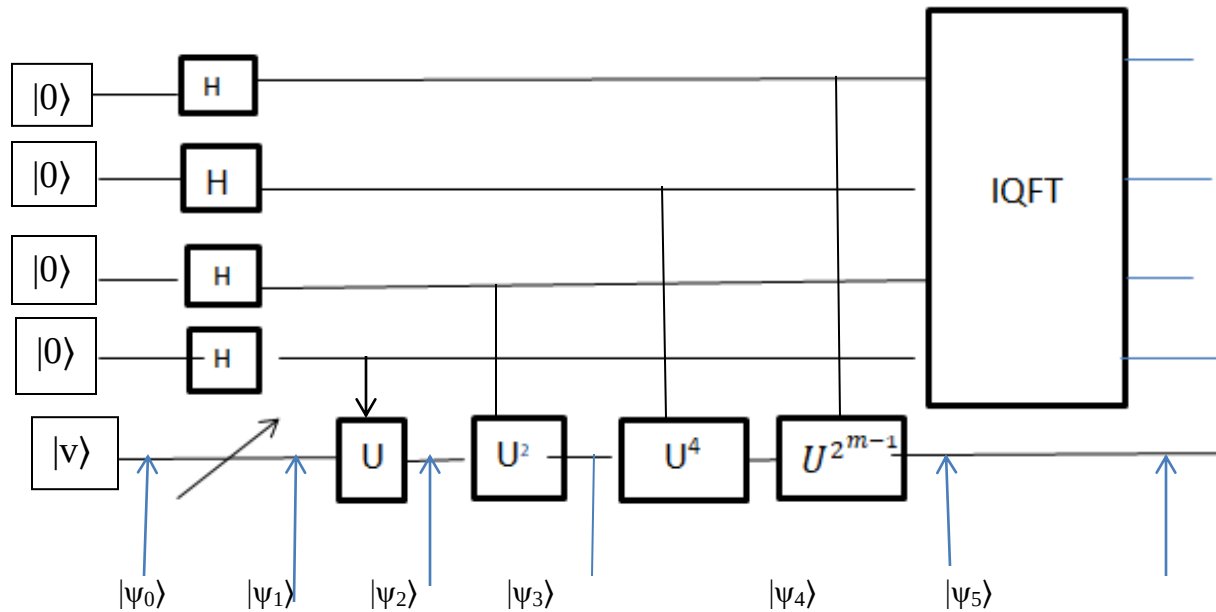
$$\text{However, } e^{\frac{-2\pi i s(0)}{r}} = e^{\frac{-2\pi i s(r)}{r}} = 1$$

$$\text{But, } |U_s\rangle = \frac{1}{\sqrt{r}} \left(e^{\frac{-2\pi i s(0)}{r}} |a \pmod N\rangle + e^{\frac{-2\pi i s(1)}{r}} |a^2 \pmod N\rangle + \dots + e^{\frac{-2\pi i s(r-1)}{r}} |a^r \pmod N\rangle \right)$$

$$\text{From this we can get, } U|U_s\rangle = e^{\frac{2\pi i s}{r}} |U_s\rangle$$

The circuit of quantum phase estimation looks like below where $|0\rangle^{\otimes m}|0\rangle$ represent the control(first) register with m qubits and $|v\rangle$ describes the work(second) register with n qubits.

The Cruiality Of Shor's Algorithm In Breaking RSA



We can find its eigenvalue($e^{i\theta}$) for which $\theta = \frac{2\pi s}{r}$ given its eigenvector $|v\rangle$.

At $|\psi_0\rangle = |0\rangle|0\rangle^{\otimes m}|v\rangle \dots \dots \dots (1.2)$

Applying Hadamard gate to all of the m qubits of the first(control) register, we get

At $|\psi_1\rangle = \frac{1}{\sqrt{2^m}}((|0\rangle + |1\rangle)^{\otimes m}|v\rangle) \dots \dots \dots (1.3)$

At $|\psi_2\rangle = \frac{1}{\sqrt{2^m}}((|0\rangle + |1\rangle)^{\otimes m} \otimes (|0\rangle + |1\rangle)^{\otimes m} \otimes \dots \otimes (|0\rangle + |1\rangle)^{\otimes m} \otimes e^{i\theta}|1\rangle) \dots \dots \dots (1.4)$

After applying Unitary gate U to all of the control register, we end up with,

At $|\psi_4\rangle = \frac{1}{\sqrt{2^m}}((|0\rangle + e^{2^{m-1}i\theta}|1\rangle) \otimes (|0\rangle + e^{2^{m-2}i\theta}|1\rangle) \otimes \dots \otimes (|0\rangle + e^{2i\theta}|1\rangle) \otimes (|0\rangle + e^{i\theta}|1\rangle) \dots (1.5)$

Using the fact that $\theta = 2\pi j$ where $0 \leq j \leq 1$ and $j = 0.j_0j_1j_2 \dots j_{m-1}$

We can rewrite $|\psi_4\rangle$ in another form

$$|\psi_4\rangle = \frac{1}{\sqrt{2^m}} (((|0\rangle + e^{2^{m-1}i(2\pi(0.j_0j_1j_2 \dots j_{m-1}))}|1\rangle) \otimes ((|0\rangle + e^{2^{m-2}i(2\pi(0.j_0j_1j_2 \dots j_{m-1}))}|1\rangle) \otimes \dots \otimes ((|0\rangle + e^{2i(2\pi(0.j_0j_1j_2 \dots j_{m-1}))}|1\rangle) \otimes ((|0\rangle + e^{i(2\pi(0.j_0j_1j_2 \dots j_{m-1}))}|1\rangle) \dots \dots \dots (1.6)$$

But from binary rule we have $j = 0.j_0j_1j_2 \dots j_{m-1} = \frac{j_0}{2} + \frac{j_1}{2^2} + \dots + \frac{j_{m-1}}{2^m}$

Replacing $0.j_0j_1j_2 \dots j_{m-1}$ by $\frac{j_0}{2} + \frac{j_1}{2^2} + \dots + \frac{j_{m-1}}{2^m}$ and multiplying by 2^{m-1} we get the following value:

$$|\psi_4\rangle = \frac{1}{\sqrt{2^m}} ((|0\rangle + e^{2\pi i(\frac{j_{m-1}}{2})}|1\rangle) \otimes (|0\rangle + e^{2\pi i(\frac{j_{m-1}}{2} + \frac{j_{m-2}}{2^2})}|1\rangle) \otimes \dots \otimes (|0\rangle + e^{2\pi i(\frac{j_{m-1}}{2^m} + \dots + \frac{j_0}{2})}|1\rangle))$$

The Cruiality Of Shor's Algorithm In Breaking RSA

The above equation shows what we would get if we applied quantum fourier transform to $|j\rangle$. for this reason by applying inverse quantum fourier transfrom(IQFT) to $|\psi_4\rangle$ we get the value of j to m bits digit.

$$\text{IQFT}|\psi_4\rangle = |\psi_5\rangle = |j\rangle \dots\dots\dots(1.8)$$

From this we can conclude that measuring $|j\rangle$ we get the decimal value of $\frac{s}{r}$.

3. using continued fraction algorithm to find r from $\frac{s}{r}$.

From measuring quantum phase estimation we get the value of $\frac{s}{r}$ which is non-repeating decimal number. To find real value of r , we need to approximate best. Let's take to coprime numbers b and c , such that $b, c < N$

$$\text{Such that, } \frac{s}{r} = \frac{b}{c}$$

Given the best approximation from the superposition of $|U_s\rangle$. Even though b and c are coprime, sometimes r and s are not coprime because of that b and c may have lost some factors that were in s and r . this can be avoided by rerunning the computer order finding subroutine an arbitrary number of times, to produce a list of fraction approximation,

$$\frac{b_1}{c_1}, \frac{b_2}{c_2} \dots \frac{b_k}{c_k} \text{ where } k \text{ is the number of subroutine was run.}$$

Each c_k will have different factors taken out of it because the circuit measured multiple different possible values of j .

To recover the actual value of r , we take the lcm of each c_k we measured.

$$r = \text{lcm}(c_1, c_2, c_3, \dots, c_j)$$

Finally, after getting the modular exponentiation period, r , we can use the concept of find $\text{gcf}(g^{r/2}+1, N)$ and $\text{gcf}(g^{r/2}-1, N)$ which are most probably a and/ or b . However, if either the gcf give N or the modular exponentiation period is odd we repeat the algorithm from choosing the initial number g , such that $g^r = 1 \pmod{N}$.

The Cruiality Of Shor's Algorithm In Breaking RSA

REFERENCES

- [1] *Shor's Algorithm* Elisa B'auumer, Jan-Grimo Sobez, Stefan Tessarini May 15, 2015
- [2] *An Efficient All-to-All GCD Algorithm for Low Entropy RSA Key Factorization*, Elijah Pelofske
- [3] *Quantum Computing: Efficient Prime Factorization*, Aaron Geelon So
- [4] *Distributed Shor's algorithm*, Ligang Xiao, Daowen Qiu, Le Luo, Paulo Mateus
- [5] *Quantum Secure Anonymous Communication Networks*, Mohammad Saidur Rahman, Stephen DiAdamo, Miralem Mehic, and Charles Fleming